20: Inference for Logistic Regression

Stat 230 - Spring 2026

```
library(dplyr)
library(broom)
library(ggformula)
library(car)
library(ggResidpanel)
```

In this tutorial we'll explore the model fit of a binary logistic regression using R. We'll use the fake news dataset as an example. To load the data, run the following code chunk:

```
# news <- read.csv("https://aluby.github.io/stat230/data/fake_news_bayesrules.csv")
news <- read.csv(here::here("data/fake_news_bayesrules.csv"))
```

We'll explore the same model discussed in class, which is fit below.

```
news_mod <- glm(type_num ~ title_words + title_has_excl + title_caps_percent,
                data = news, family = "binomial")
```

In the next sections, you'll be guided through conducting inference and creation of the various diagnostic plots discussed in class. Be sure to run each code chunk to see the resulting plots, and don't hesitate to ask me questions if things are unclear!

1 Inference

You have already seen that we can obtain Wald-based tests via the `summary()` command. To extract only that coefficient table, you can use the `tidy()` command from the broom package:

```
library(broom)
tidy(___)
```

To calculate Wald-based confidence intervals for each regression coefficient, we can use the `confint()` command, specifying the level argument:

```r
tidy(news_mod, conf.int = TRUE, conf.level = 0.9)
```

```
# A tibble: 4 x 7
  term               estimate std.error statistic  p.value conf.low conf.high
  <chr>                 <dbl>     <dbl>     <dbl>    <dbl>    <dbl>     <dbl>
1 (Intercept)         -2.23      0.672      -3.32 0.000894 -3.38      -
1.16
2 title_words          0.120     0.0581      2.07 0.0383    0.0260     0.218
3 title_has_exclTRUE   2.00      0.849       2.35 0.0186    0.722      3.61
4 title_caps_percent   0.0498    0.0295      1.69 0.0908    0.00194    0.0993
```

2 Checking conditions

Like in linear regression, we can make plots to assess the fit of our binary logistic regression model. The interpretation of these plots is harder, however, since the response variable is binary; thus, we can't expect to see a random scatter of points around zero like we did in linear regression. We'll use two tools:

1. **Empirical logit plots** to check the *linearity* condition (is the log odds a linear function of each predictor?)
2. **Pearson residual plots** with a LOESS smoother to check linearity *after adjusting for the other predictors*, and to look for outliers and patterns over time

2.1 Empirical logit plots

To construct an empirical logit plot for a quantitative predictor we follow these steps:

1. Order the observations by the predictor and use the ordered data to create g bins of approximately equal size. (A reasonable starting point is $g = \sqrt{n}$)
2. In each bin, count the number of successes ($Y = 1$) and failures ($Y = 0$)
3. Calculate the empirical logit in each bin: $\log\left(\frac{\#\text{successes}+0.5}{\#\text{failures}+0.5}\right)$. (The 0.5 keeps us from taking the log of zero in a bin with no successes or no failures.)
4. Plot the empirical logits against the mean value of the predictor in each bin
5. Look for a straight-line pattern. A curved pattern suggests that the predictor needs a transformation

Let's implement these steps for `title_words` using 13 bins. First, we create the bins and summarize each one:

```r
words_el <- news |>
  mutate(bin = ntile(title_words, 13)) |>
  group_by(bin) |>
  summarize(mean_x = mean(title_words),
            n = n(),
            successes = sum(type_num),
            failures = n - successes,
            emp_logit = log((successes + 0.5) / (failures + 0.5)),
            .groups = "drop")

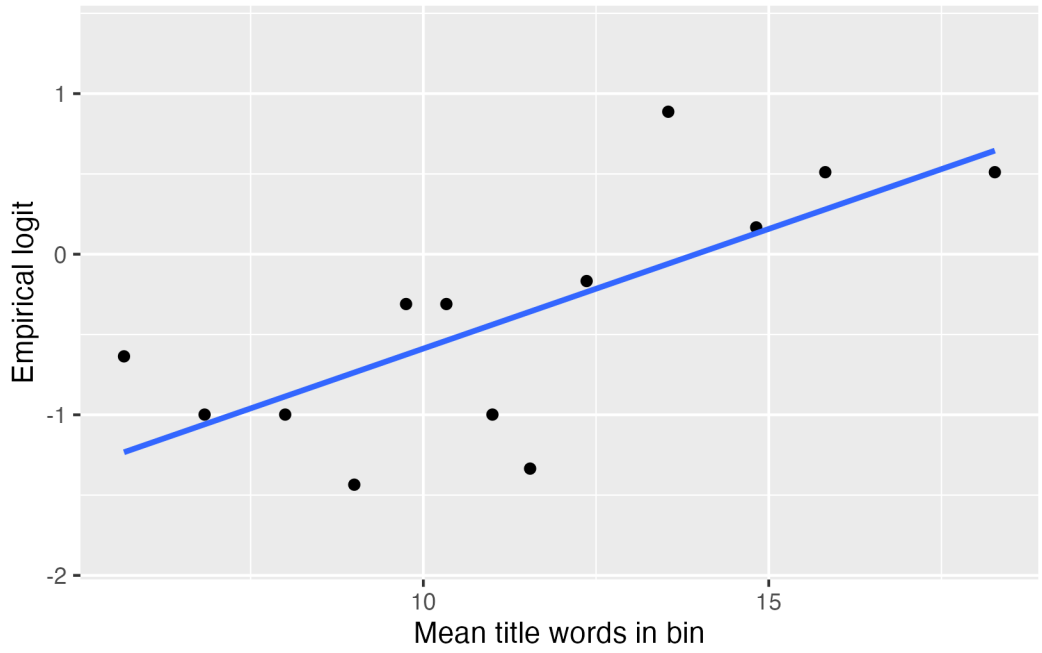
words_el
```

```
# A tibble: 13 x 6
     bin mean_x     n successes failures emp_logit
   <int>  <dbl> <int>     <int>    <int>     <dbl>
 1     1   5.67    12         4        8    -0.636
 2     2   6.83    12         3        9    -0.999
 3     3   8       12         3        9    -0.999
 4     4   9       12         2       10    -1.44
 5     5   9.75    12         5        7    -0.310
 6     6  10.3     12         5        7    -0.310
 7     7  11       12         3        9    -0.999
 8     8  11.5     11         2        9    -1.34
 9     9  12.4     11         5        6    -0.167
10    10  13.5     11         8        3     0.887
11    11  14.8     11         6        5     0.167
12    12  15.8     11         7        4     0.511
13    13  18.3     11         7        4     0.511
```

> **Adapting the code**
>
> To adapt this code for your own binary logistic regression model, replace news with the name of your data set, `title_words` with your predictor (in both `ntile()` and `mean()`), and `type_num` with your 0/1 response. You can adjust the number of bins (currently 13) based on your data size.

Next, we plot the empirical logits against the mean predictor value, and add a straight line to help us judge whether the pattern is linear:

```
gf_point(emp_logit ~ mean_x, data = words_el,
         xlab = "Mean title words in bin", ylab = "Empirical logit") |>
  gf_lm()
```



Each point is based on only about 11 articles, so expect some scatter around the line. Look for the overall trend, not for individual points.

2.2 Categorical predictors

For a categorical predictor, we calculate the empirical logit for each level of the predictor (no bins are needed):

```
news |>
  group_by(title_has_excl) |>
  summarize(n = n(),
            successes = sum(type_num),
            failures = n - successes,
            emp_logit = log((successes + 0.5) / (failures + 0.5)))
```

```
# A tibble: 2 x 5
  title_has_excl     n successes failures emp_logit
  <lgl>          <int>     <int>    <int>     <dbl>
```

```
1 FALSE            132         44         88    -0.688
2 TRUE              18         16          2     1.89
```

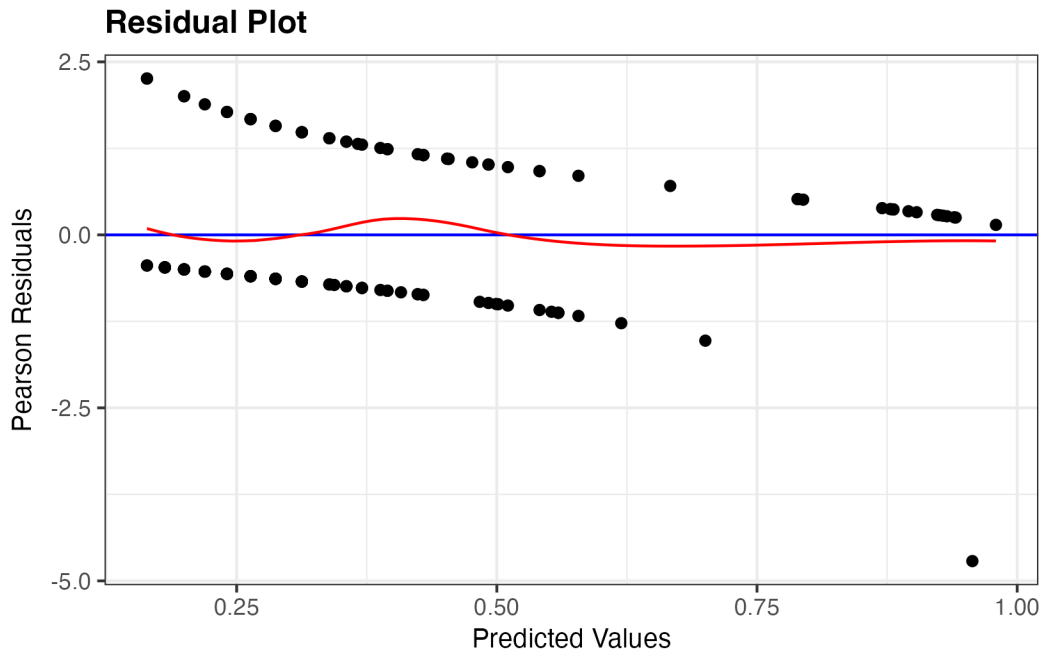
With only two levels, the two points always fall on a straight line, so there is no linearity condition to check for a binary predictor. These plots are more useful for *comparing* groups (for example, to decide whether an interaction is needed).

3 Pearson residuals

To create a Pearson residual plot with a LOESS smoother, we can use the `resid_panel` command from the `ggResidPanel` Package, similar to what we did in linear regression. The only difference is that we need to specify `type = "pearson"` to indicate that we want to plot Pearson residuals. For binary data the points fall in two curved bands (one for each value of Y), so the **smoother** is what we look at. If the model is adequate, it should be roughly a horizontal line at 0.

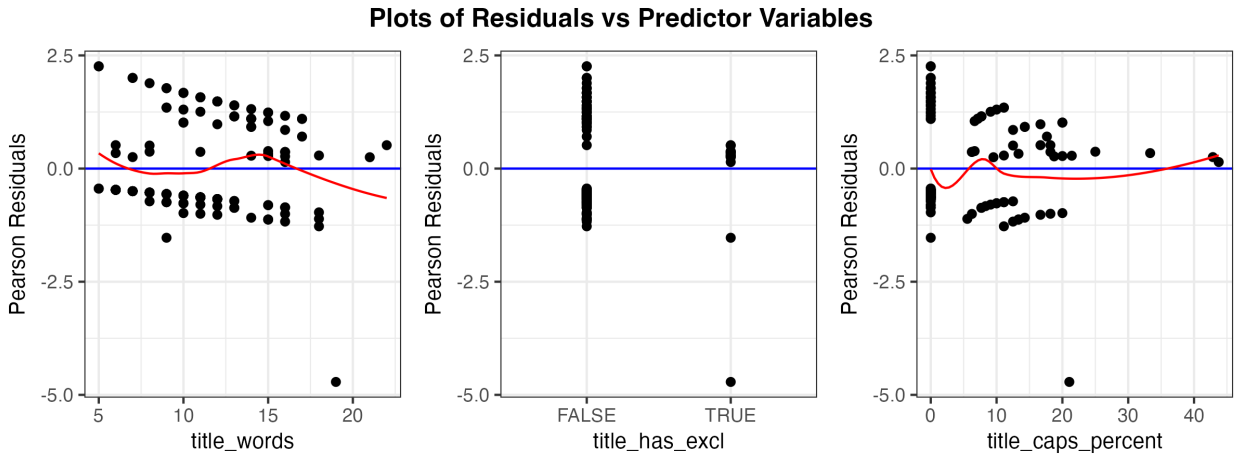
Below we recreate the residual plots from the slides:

```
resid_panel(news_mod, plot = "resid", type = "pearson", smoother = TRUE)
```



4 resid_xpanel

```
resid_xpanel(news_mod, yvar = "residual", type = "pearson", smoother = TRUE, nrow = 1
```



5 Your turn

- Make an empirical logit plot for `title_caps_percent`. Start with 13 equal-size bins using `ntile()` and look at the `mean_x` column of your summary. What do you notice? Why is this a problem?
- Now create the bins by hand with `cut()`, for example `mutate(bin = cut(title_caps_percent, breaks = c(-Inf, 0, 10, 15, 20, Inf)))`, and make the plot again. What do you conclude about the linearity of `title_caps_percent`?
- Look at the Pearson residual plots above. Do they agree with your empirical logit plots? Is there any evidence of a violation of the linearity condition?
- Report 95% confidence intervals and provide interpretations on the **odds** scale for all predictor variables
- Discuss together: why do we need to make plots and check conditions?

---

6 Function quick reference

The following table summarizes the functions discussed above:

Function	Purpose
`mutate(bin = ntile(x, g))`	Divide observations into g bins of (about) equal size, ordered by x
`cut(x, breaks)`	Create bins from specific cut points you choose
`group_by(variable)`	Group data by variable for summarization
`summarize()`	Summarize data by calculating statistics like mean, count, etc.
`log((successes + 0.5) / (failures + 0.5))`	Empirical logit
`gf_point(y ~ x, data, xlab, ylab)`	Create scatter plot of y vs. x with labels
`gf_lm()`	Add a straight line to a scatterplot
`resid_panel(model, plot = "resid", type = "pearson", smoother = TRUE)`	Create Pearson residual plot with LOESS smoother
`resid_xpanel(model, type = "pearson", smoother = TRUE)`	Pearson residuals against each predictor